

FrutLog

Relatorio tecnico detalhado do backend

Analise de implementacao, arquitetura, dados, seguranca e pontos de evolucao

ESCOPO ANALISADO

server.js, api/[...route].js, scripts/password-hash.js, README.md e migracoes SQL em supabase/.

DATA DA ANALISE

23 de setembro de 2026

Resumo executivo

O backend e uma API Node.js sem framework, com Supabase/PostgREST como persistencia e uma rota dedicada para ingestao de telemetria IoT. A base implementa controles importantes: senha PBKDF2, JWT assinado, comparacao em tempo constante, CORS por allowlist, validacao de entrada e limite basico por IP.

Conclusao: a solucao tem um nucleo adequado para um MVP, mas apresenta uma divergencia critica entre a API e o esquema de banco mais recente. Antes de producao, e indispensavel concluir a migracao/adaptacao do modelo de dados e corrigir as rotas operacionais que ainda apontam para as tabelas legadas.

Classificacao geral

Dimensao	Leitura
Arquitetura	Boa separacao entre front-end, API, ponte MQTT e banco.
Seguranca	Boas praticas presentes; faltam endurecimentos para escala e operacao.
Dados	Modelo operacional mais rico existe, mas a API esta parcialmente desalinhada.
Prontidao	MVP controlado; nao recomendado para producao sem plano de correcao.

1. Arquitetura e fluxo

O servidor HTTP em server.js entrega arquivos estaticos do diretorio do front-end e encaminha caminhos iniciados por /api para o handler central. O handler usa somente modulos nativos do Node.js: crypto para autenticacao e fetch para o PostgREST do Supabase.

Fluxo principal de telemetria

1. Sensor	2. Ponte privada	3. API FrutLog	4. Supabase
Publica no topico MQTT com TLS.	Assina, valida topico e JSON.	POST /api/iot/telemetria com chave.	Persiste leitura e atualiza dispositivo.

A ponte MQTT descrita no README e corretamente posicionada fora de uma funcao serverless: ela requer conexao persistente, assinatura controlada e credencial de leitura separada. O navegador nao recebe credenciais de broker nem chave IoT.

Rotas identificadas

Metodo e rota	Protecao	Responsabilidade
GET /api/health	Publica + CORS	Verifica acesso basico a organizacao no banco.
POST /api/login	Limite 10/min por IP	Valida matricula/senha e emite JWT de 8 horas.
POST /api/iot/telemetria	X-IoT-Key	Autoriza dispositivo/sensor e grava leitura.
GET /api/painel-*	JWT	Monta dados de talhoes, sensores, leituras e inspecoes.
POST /api/plantios	JWT: engenheiro/admin	Registra plantio.
POST operacionais	JWT: tecnico/admin	Inspecoes, ocorrencias e problemas de sensor.
GET/POST admin/usuarios	JWT: admin	Lista e cria usuarios.

2. Autenticacao e seguranca

A autenticao e implementada diretamente com primitives criptograficas do Node.js. Senhas sao armazenadas em formato pbkdf2\$iteracoes\$salt\$hash, usando PBKDF2-SHA-256, salt aleatorio de 16 bytes e 210.000 iteracoes. A verificacao usa comparacao em tempo constante.

Controles ja presentes

Controle	Implementacao observada	Avaliacao
JWT	HMAC-SHA256; claims de usuario; validade de 8h.	Adequado para sessao simples.
CORS	Somente FRONTEND_ORIGIN e localhost local; OPTIONS restrito.	Boa allowlist.
IoT	Chave em tempo constante; dispositivo, sensor e talhao verificados.	Bom controle de ingresso.
Entrada	JSON limitado a 16 KiB; texto, datas, tipos e valores validados.	Boa defesa inicial.
Cabecalhos	nosniff e no-store nas respostas JSON.	Positivo.
Banco	Service role no servidor; RLS nas migracoes.	Exige isolamento rigoroso.
Rate limit	Map em memoria, 10/min login e 120/min demais rotas.	Limitado a instancia unica.

Riscos e recomendacoes

- Rate limiting: trocar o Map em memoria por Redis, Upstash ou controle no gateway/reverse proxy.
- Proxy: confiar em x-forwarded-for apenas atras de proxy conhecido; hoje um cliente pode forjar IPs.
- JWT: nao ha revogacao nem refresh; usuarios desativados mantem token valido ate expirar.
- Segredos: rotacionar IOT_API_KEY e manter todas as chaves exclusivamente no ambiente do servidor.
- Auditoria: registrar criacao de usuario, login, negacoes de autorizacao e alteracoes operacionais.

3. Dados e integracao com Supabase

O repositório contém dois mundos de dados. `supabase/schema.sql` descreve tabelas legadas pluralizadas (`usuarios`, `talhoes`, `dispositivos_iot`, `sensores` e `telemetrias`). As migrações 002-005 introduzem um modelo operacional singular e mais normalizado.

Modelo operacional previsto

Camada	Entidades e finalidade
Contexto	organizacao, fazenda, talhao: delimitam propriedade e localizacao.
Cultivo	cultura, cultivar, ciclo_cultura, colheita: historico agricola e produtividade.
IoT	dispositivo, sensor, leitura_sensor: equipamento, metrica e serie temporal.
Operacao	inspecao, regra_alerta, alerta, evento_auditoria: acompanhamento e rastreabilidade.

Integridade e desempenho

A migração 003 adiciona índices para dispositivos, sensores, alertas e ciclos; cria unicidade para regras de alerta; e instala um trigger que impede leitura associada a sensor de outro dispositivo. A migração 005 traduz dados legados sem apagar as tabelas antigas.

Divergencias criticas identificadas

Evidencia	Impacto	Acao recomendada
Ingestao IoT usa dispositivo, sensor e leitura_sensor.	Depende das migracoes 002/003/005.	Manter e testar contra banco migrado.
Login/admin usam usuarios; painel usa talhao/sensor.	A aplicacao mistura contratos.	Padronizar API para o modelo singular.
Plantios e rotas tecnicas usam tabelas plurais; painel consulta inspecao singular.	Registros podem nao aparecer no painel.	Mapear payloads e substituir tabelas legadas.
README menciona tabelas antigas; IoT atual usa nomes singulares.	Provisionamento incorreto.	Atualizar README e exemplos.

4. Qualidade, operacao e evolucao

O projeto e deliberadamente enxuto: nao declara dependencias externas e centraliza a logica em um handler. Isso reduz superficie inicial, mas torna a manutencao mais dificil quando crescer. Nao foram localizados testes automatizados do backend, esquema de validacao reutilizavel, logger estruturado ou observabilidade.

Pontos tecnicos a observar

Tema	Observacao
Tratamento de erros	A funcao db protege detalhes do cliente, mas nao registra status/corpo do PostgREST para diagnostico. Adicionar logs estruturados sem segredos.
Consultas do painel	Busca ate 500 leituras e monta joins em memoria. Para volume maior, mover agregacoes para view/RPC SQL paginada.
Requisicoes SQL	Queries sao montadas por concatenacao com encodeURIComponent. Um cliente Supabase ou RPC reduz fragilidade e melhora tipagem.
Cabecalhos HTTP	Avaliar CSP, HSTS em HTTPS e X-Frame-Options na infraestrutura.
Static files	Protege contra path traversal; no-cache e adequado ao desenvolvimento. Em producao, usar assets com hash.

Plano priorizado

Prioridade	Entrega	Criterio de aceite
P0 - antes de producao	Unificar API e schema operacional; remover dependencia das tabelas plurais.	Fluxo completo em banco migrado.
P0 - antes de producao	Corrigir documentacao IoT e validar migracoes em homologacao.	README, SQL e rotas com o mesmo contrato.
P1	Testes de login, RBAC, telemetria, validacao e painel.	Suite automatizada em CI.
P1	Observabilidade e rate limiting distribuido.	Logs com request-id, metricas e limite compartilhado.
P2	Refatorar handler em modulos.	Responsabilidades separadas e testaveis.

Conclusao final

A arquitetura proposta para o caminho IoT e sensata, e varios controles essenciais ja estao codificados. A maior fragilidade e a coexistencia de dois modelos de dados sem uma adaptacao completa da API. Resolver essa fronteira, validar o fluxo completo e estabelecer testes darao ao backend uma base muito mais confiavel para evoluir.